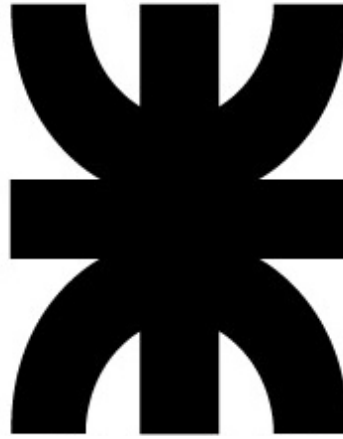


Universidad Tecnológica Nacional

Facultad Regional de Córdoba



Ingeniería en Sistemas de Información

Ingeniería y calidad de software

Tema: TP 6 - Test Driven Development (TDD)

Año 2025

Curso: 4K2

Grupo Nro: 1

Docentes:

- Ing. Meles Silvia Judith
- Ing. Massano María Cecilia

Integrantes:

- Lerchundi Agustina - 90105
- Bustos Joaquin 94914
- Castoldi Thiago Martin - 94986
- Noto Claudia Carina - 95215
- Romero Moreno Oscar Alfonso - 96454
- Spadachini Martin Matias - 95168
- Sadir Emilio - 96622
- Petrich Ernesto Joaquin - 90431
- Filippa Franco Julian - 92191
- Recalde Franco - 94661

Descripción de la US	3
User Story	3
TDD: Calcular monto	3
Justificación de Lenguaje	7
Backend	7
Frontend	7
Estilo de Código y Estándares	8
Bibliografía	9

Descripción de la US

User Story

“Comprar entrada”

“COMO visitante QUIERO comprar una entrada PARA asegurar mi visita al parque”

Criterios de aceptación:

- Debe indicar la fecha de visita deseada, la cantidad de entradas requeridas, la edad de cada visitante y tipo de pase (VIP o regular).
- La fecha de visita guiada puede ser del día actual o futuro.
- Debe enviar un mensaje de confirmación vía mail.
- Debe redirigir a Mercado Pago al confirmar la compra si el pago es con tarjeta de crédito.
- La fecha de la visita debe estar dentro de los días en que el parque está abierto.
- Debe seleccionar la forma de pago: efectivo en caso de querer pagar en boletería o con tarjeta.
- La cantidad de entradas requeridas no debe ser mayor a 10.
- Al finalizar la compra se debe informar la cantidad de entradas compradas y la fecha.
- Se debe permitir la compra de entradas solo a usuarios registrados.

Pruebas de Usuario:

- Probar comprar una entrada indicando la fecha de visita dentro de los días disponibles, una cantidad de entradas menor a 10, la edad de todos los visitantes, el tipo de pase, la forma de pago con tarjeta mediante Mercado pago y recepción del mail de confirmación. **[PASA]**
- Probar comprar entradas ingresando una fecha de visita en la cual el parque se encuentra cerrado. **[FALLA]**
- Probar comprar entradas sin seleccionar forma de pago. **[FALLA]**
- Probar comprar entradas ingresando una cantidad de entradas mayor a 10. **[FALLA]**

Test: Calcular monto

// PRECONDICIONES

1. Crear usuario de prueba en la base de datos

usuario = {

```
'nombre': 'Test User',  
'email': 'test_user@example.com',  
'hashed_password': 'fake_hashed_password_123',  
'estado': 'activo'  
}  
  
insertar_usuario_en_db(usuario)
```

2. Mock de funciones de seguridad

```
mockear_funcion('hash_password', retorna='fake_hashed_password_123')  
  
mockear_funcion('verify_password', retorna=True)
```

3. Datos de entrada del test (credenciales válidas)

```
payload = {  
  
    'email': 'test_user@example.com',  
  
    'password': 'password_secreta'  
}
```

4. Configuración de seguridad JWT

```
SECRET_KEY = 'clave_secreta_simulada'  
  
ALGORITHM = 'HS256'
```

5. Sesión activa y endpoint disponible

```
cliente = iniciar_cliente_api('/auth/login')  
  
endpoint_disponible = verificar_endpoint('/auth/login')
```

6. Resultado esperado

Se espera que el sistema devuelva:

- Código HTTP 200
- JSON con “access_token” y “token_type” = “bearer”
- Token JWT válido con claim “sub” igual al email del usuario

//PASOS DEL CASO DE PRUEBA

1. El usuario ingresa al sistema autenticado como "Visitante"
`iniciar_sesion('usuario_registrado', 'password_segura')`
2. El usuario selecciona la opción del menú "Comprar entradas"
`seleccionar_opcion_menu('Comprar entradas')`
3. El sistema solicita los datos de los visitantes
`ingresar_datos_visitantes([
 {'edad': 20, 'tipo_entrada': 'VIP'},
 {'edad': 25, 'tipo_entrada': 'VIP'}
])`
4. El usuario confirma la cantidad de entradas y continúa con el cálculo del monto total
`confirmar_datos_compra()`
5. Llamado a la Historia de Usuario principal (US)

Funcionalidad: calcular el monto total de la compra según el tipo de pase y edad.

`calcular_monto(
 visitantes = [
 Visitante(20, VIP),
 Visitante(25, VIP)
]
)`
6. El sistema procesa los datos, aplica las reglas de negocio

- VIP = \$10.000

- REGULAR = \$5.000

- Menores de 3 años = \$0

- Mayores de 3 y menores de 15 años = precio * 0,5

- Mayores de 60 años = precio * 0,5

- Máximo de 10 visitantes permitidos

7. El sistema muestra el monto total calculado

mostrar_resultado("El monto total a pagar es \$20000")

8. Validación del resultado

assert monto == 20000

// RESULTADOS

monto_calculado = calcular_monto([

Visitante(20, VIP),

Visitante(25, VIP)

])

assert '20000' == xpected_monto, \

"El monto calculado es {20000}, pero se esperaba {20000}"

Justificación de Lenguaje

Backend

El desarrollo del sistema para la organización EcoHarmony fue a realizado a través de Python, basándonos expresamente en los documentos de **PEP 20: The Zen of Python**, esto ya que se buscaba alcanzar una serie de objetivos dentro del proyecto que nos ayuden a mantener y reforzar diversos atributos que mejoren la calidad, no solo del código sino también del trabajo grupal.

El Zen of Python establece un conjunto de sentencias que buscan orientar la escritura de software a algo legible y coherente. Dicho documento contempla 19 principios y fue escrito en 1999 por Tim Peters, estos son:

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Flat is better than nested.
- Sparse is better than dense.
- Readability counts.
- Special cases aren't special enough to break the rules.
- Although practicality beats purity.
- Errors should never pass silently.
- Unless explicitly silenced.
- In the face of ambiguity, refuse the temptation to guess.
- There should be one-- and preferably only one --obvious way to do it.
- Although that way may not be obvious at first unless you're Dutch.
- Now is better than never.
- Although never is often better than *right* now.
- If the implementation is hard to explain, it's a bad idea.
- If the implementation is easy to explain, it may be a good idea.
- Namespaces are one honking great idea -- let's do more of those!

Frontend

El desarrollo del frontend fue desarrollado a través de un página web y se implementó utilizando React, una biblioteca de JavaScript creada por Facebook para la construcción de interfaces de usuario dinámicas, modulares y reactivas.

La elección de React se fundamenta en su filosofía declarativa, su arquitectura basada en componentes reutilizables y su fuerte enfoque en la experiencia del usuario (UX), que se alinea con los principios de diseño limpio y claridad estructural también presentes en el backend desarrollado con Python.

Un punto fuerte en la decisión de emplear esta tecnología para el desarrollo se fundamenta sobre “The React Philosophy”, que actúa como declaración de principios de diseño de la biblioteca, basándose en cuatro puntos:

- Componentización
- Unidirectional Data Flow
- Declarative Programming
- State-driven UI

Estilo de Código y Estándares

El desarrollo del proyecto se llevó a cabo siguiendo principios de legibilidad, mantenibilidad y consistencia, tanto en el backend (con Python) como en el frontend (con React, JavaScript).

El cumplimiento de normas de estilo y convenciones de codificación no solo favorece la comprensión del código por parte del equipo, sino que también mejora la calidad del software y reduce errores durante la evolución del sistema.

Mientras que en el sistema general se aplicó un único lineamiento a la hora de declarar las diversas variables y métodos, utilizando *snake_case*, como por ejemplo: “**test_calcular_monto_ok**” o “**test_calcular_monto_error**”. Sin embargo, en caso especiales se hizo uso de *UPPER_CASE*, como a la hora de simular valores: “**TEST_PASSWORD**” o “**TEST_EMAIL**”.

A su vez, haber planteado el programa bajo estos estándares nos asegura que se presentan diferentes características las cuales son deseadas a la hora de programar, como, por ejemplo:

- Legible y coherente entre los distintos módulos.
- Fácil de mantener y escalar por nuevos desarrolladores.
- Alineado con buenas prácticas de ingeniería de software moderna.

Dentro del Frontend podemos encontrar que los estándares se ajustan a lo descrito en el Airbnb JavaScript Style Guide (2024).

Este documento es uno de los lineamientos más adoptados en la industria para garantizar uniformidad y buenas prácticas.

Entre sus principios más relevantes aplicados al proyecto se destacan:

- Uso de componentes funcionales con Hooks (useState, useContext, useNavigate).
- Convención de nombres en PascalCase para componentes (CompraEntradas, ModalPago) y en camelCase para funciones y variables (handleCancelar, setMostrarModal).
- Estructura declarativa y predecible: se prioriza el “qué” sobre el “cómo”.
- Uso consistente de comillas dobles y constantes para estados inmutables.
- Separación de responsabilidades: la lógica del estado, la renderización y los estilos visuales se manejan de forma clara y separada dentro del componente.

Aparte de eso se tuvo en consideración la opinión particular del product owner el cual especificó el tipo de letra y la paleta de colores que se debían usar en la aplicación.

Buenas ! La paleta de colores que seleccionamos fue la siguiente:

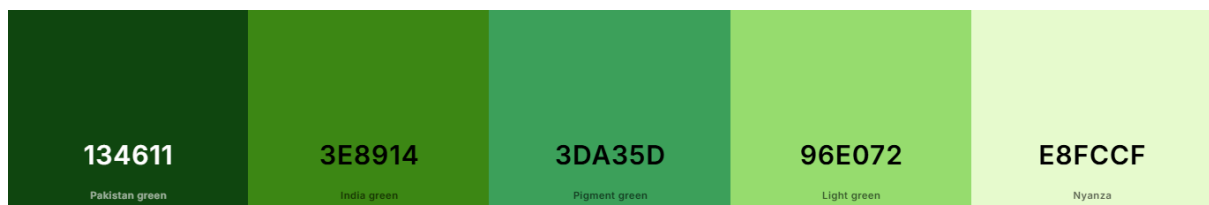
<https://coolors.co/134611-3e8914-3da35d-96e072-e8fccf>

El tipo de letra a utilizar debe ser Montserrat.

El icono no está definido, pueden volverse creativos con eso !

Muchas gracias por la consulta !

Img 1. Comunicación con el Product Owner



Img 2. Paleta de colores definida por el Product Owner

Bibliografía

1. Van Rossum, G., Warsaw, B., & Coghlan, N. (2001). *PEP 8 – Style Guide for Python Code*. Python Software Foundation.
2. Peters, T. (2004). *PEP 20 – The Zen of Python*. Python Software Foundation.
3. Airbnb Engineering & Data Science. (2024). *Airbnb JavaScript Style Guide*. GitHub repository.
4. React Team. (2024). *React Documentation & Thinking in React*. Meta Open Source.

5. ECMA International. (2023). *ECMAScript® 2023 Language Specification (13th Edition)*.
6. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.